

Депонируемые материалы

Идентификация программы для ЭВМ «VAST»

Материалы подготовлены как проект для электронной подачи. Они описывают состав исходного текста, контрольные суммы и репрезентативные фрагменты, идентифицирующие программу.

Снимок программы

Параметр	Значение
Объект	Программный комплекс VAST
Назначение	Воспроизводимое планирование, запуск и анализ экспериментов по видеоаналитике КПП.
Снимок исходников	bfce1d7 2026-06-30 V3; рабочее дерево: чистое
Состав депонирования	54 файлов; 692067 байт
Исключены из объема	build, runs, .cache, .venv, data, models, __pycache__, внешние датасеты, сгенерированные артефакты
Файлы контроля	source_manifest.csv; source_checksums.sha256

Состав по типам файлов

Тип	Файлов	Байт
.c	1	8519
.cpp	1	41797
.css	1	7013
.cu	1	37064
.deepstream	1	1424
.html	1	1870
.js	1	39976
.md	4	30083
.policy	1	153
.ps1	1	3221
.py	25	412096

.savant	1	1591
.sh	5	68990
.sha256	1	88
.txt	2	3432
.yaml	4	21104
.yml	3	13646

Перечень исходных файлов и контрольные суммы

Путь	Байт	SHA-256
CMakeLists.txt	3354	663b6722caf27290 e482374c07d3ea58 de7357cf4bd33011 c62dea10c1ffb87b
configs/datasets.yaml	11423	169caf2f61dbae2e 0809a35d676e22f2 fbceef20706a4a85 cdcefc2e4820866c
configs/experiments.yaml	7299	93db41338c7b3690 164c5eff04ee06de c0fe8f18fb0b60d4 6d49bcc19e492567
configs/hosts.example.yaml	1191	23d1e0bae98f58f2 19a0eb371ce0eaca 6845b3d739d560bb c23b463c8c9111c2
configs/hosts.yaml	1191	23d1e0bae98f58f2 19a0eb371ce0eaca 6845b3d739d560bb c23b463c8c9111c2
deploy/custom_cpp_cuda_qt/adaptive_scheduler_app.cu	37064	807127a5dc2f0374 10dba3f8589c4f1e e1ba21e5c7639a17 3e0988033652d672
deploy/gstreamer_adaptivescheduler/gstadaptivescheduler.c	8519	c94ebeabec8ed71b a8200ef2e480381a ea3b5c18606f0812 6432c38f6a4a1c33
deploy/native_gst_probe/Dockerfile.deepstream	1424	f750fead613f45cc dcf1067be5283c0d f51347ecc182fd9b 1c2f2ae346b0c34f
deploy/native_gst_probe/Dockerfile.savant	1591	95b5335fb8ff71d4 52cf3e7b769c621d cc83b8e1cf6ba7c7 ec1f041b856bdf68
deploy/native_gst_probe/vast_native_gst_probe.cpp	41797	ec9fbffa8421e396 89cd0e9c60eb7dee 20ece2d8c3070f8b 9c7ae2b22bcdbe6e
deploy/savant/canonical_distributed_module.yml	2563	d90d1c8ef07087dd 43c4c0f07ab724b9 9c6745867c1f23d2 0e9cfa24ea11b40a
deploy/savant/canonical_heterogeneous_module.yml	9547	2037010bb05b2f34 a48e0469c8336e9b 65fbf2f30481fa13 561cb4954ca83d63
deploy/savant/module.yml	1536	79e013f0f3312809 7c55c17731f5cfa5 810a5b89b1591a91 62e16de2529eca07
deploy/savant/native_probe.py	21515	a429340d77583c0f 3ade7d877f7660a5 a2321aa110bb8d8a 2b1638abd183f6c6
docs/DATASETS.md	2475	46022ca5d769d238 5bcc60a97e035575 c0c0aaceea605e40 6e6fdbdc7c316714

docs/NATIVE_ADAPTERS.md	6124	5dd27691a57e594b 819288e893dfa98e 2170ddeefd200415 92d00859576019d6
INSTRUCTIONS.md	12132	469e647637c47b8b 12a57477dc4b491e f026c985454f270d a059c9e6dcea5f7f
policies/ql_heft_frozen.policy	153	a84af2bf720d05a1 0bd9feda32150d09 8ffdd92e5108b40b 1d3e7dcd3c8a5545
policies/ql_heft_frozen.policy.sha256	88	5d0b967f4586a70f 56afd00204b1c1e3 3ac47e44efe8bdef 44c1b6cafd1df35c
README.md	9352	10b123a524815188 fd919d770a89b57f 7d01b66201d8f181 fd58ab5e89562212
requirements.txt	78	2a7a26508f0b4824 e510f266274b363b 4f377e65f67ad4ef 0382f643de548641
scripts/analyze_results.py	5062	0c2681a767d6beee 8f9a0ae8a364b7af 5a3798c4d35acecc 886798409075255d
scripts/benchmark_adapters.py	3965	eb8c07870de7639f a696c8401d28f8d4 ba80f804553d4bc0 4dfc6c3112d73ea8
scripts/benchmark_contract.py	30222	75f2d2c1dbb49b2d 0bac231b08e4b601 db34b5a1b665f6c8 e625f8e04f247de1
scripts/build_native_probe_images.sh	2541	8cd5202672aab6ba 263eded091d9a8da 1b9cd1562d244f28 d9d129e23bb2e8fc
scripts/check_dataset.py	1018	54f07366c27cd94f deaa75391f204047 b6ce3fb2839579d6 3b52ac9452aef68d
scripts/check_system.py	1501	57232b17147ceadf 576039ba7a0845aa 6e616e0258cefd4b b9ebb4266d69c544
scripts/collect_metrics.py	4808	1c565b7ff7702f58 d27c3febf6c7d84f 2155c1f25ba450fd 29e2bdb0a9f73d24
scripts/custom_app_emitter.py	2478	52c1c5528f92931b 315d7f0fb6bd462a 2045cdb665fe8b48 7c8e552c202a8c3d
scripts/distributed_executor.py	21043	edcbf3b8da72e442 bf2a9f425c9c0dd8 e5c5e9ed21bf75fe 002e1c37f17b25e9
scripts/emit_runtime_frames_csv.py	3837	e495245e5ad7058e 4a4ba996462214b1 27e1cbbaf073e922 d5f0ec186517f5f1
scripts/generate_vast_report_artifacts.py	47052	cab59e2f17d00ea5 2cb786bed30e648b 89893d7bbfe25b49 787b79b09907e429
scripts/install_openvino_dlstreamer.sh	8635	eed9120f27cc8ebf 74fb35ac0621162b 9b091cbb5d91a33a 2a5e8d51e8b1cd97
scripts/prepare_assets.sh	3789	d384dc0edaedddd d7ae81beec18f830 54e31a78698dcf55 ba918d07cda9e50d
scripts/prepare_benchmark_dataset.py	9832	41cc05d1334800e2 2f67981105bec409 c63c2c10aff8d12d 9c86b88632ffa704
scripts/rtp_trace.py	1497	47f94ac962938a2e ce2fd4493ec432b8 93a46e676cc66ac6 39756bda07498f35
scripts/run_experiments.py	54081	1d313daa0bad11ab 71c5833b18a7ed93 cef012d53ca99de6 93adcb5ebebe1521
scripts/run_openvino_container_chunks.py	10880	343549a85047bae6 d1f2282058b637df 4d8adfc2ceb288b

		45db7ddb2038f00d
scripts/run_system_template.sh	43450	4728cf1d5da4c1ac 21f8ffdbee8bc7b2 7bbe282d86e38a3e 27f4fa03c938710c
scripts/setup_target.py	1805	098c5f771af92bb1 1322e1a49ef2bd3a 89b1ddeb24b93fe3 ab3dc2d9d9943525
scripts/setup_target.sh	10575	fed1843edf535464 e3da6b25e73da863 1c4a6b6e88e71ee3 27fbe7c5c467d3bb
scripts/setup_target_windows.ps1	3221	176768a9d802a191 5b5c74c885013450 669549f72dd9296a cb63b61f5e1852f3
scripts/train_ql_heft.py	2279	d7183f5b543a437b cfed0b8bf0f0df9c 293317ee5b4479a6 0596f3920b7a37fe
scripts/validate_strict_telemetry_fix.py	15807	7495b73b8d85887a 46da9be3dc4d769c 06c1637b5f490319 b8861ada5ea89856
scripts/vast_gui.py	49690	95b2c559eb8a9a75 150495a47a421296 8a044b193c492026 8172e9d3a6b1bd49
tests/test_benchmark_contract.py	42548	9ab2744845b264e3 079d3406eb299b0d 0f39ac0ee5ba1ac3 cb96c0364bc2717c
tests/test_prepare_benchmark_dataset.py	7439	6b876bfd02e661de 9754f4bc586be571 adb639ec15646cfd 3f748b0afc00358d
tests/test_run_experiments_resume.py	3486	b48e898fd3499277 a71f294c4c48ca81 da776f1cb681512e 8f1b43b2f1a4b4d5
tests/test_scenario_planning.py	39481	8fcc22b049d0c34c 5b4ac48f821a1e9f db58aebf0156fe5a 9aaf08ad110d6fa0
tests/test_strict_validation.py	16073	245a98c81bba518c 8424ac725534b425 f4e4d85f0c7b1399 cb6386f3eca8a987
tests/test_vast_gui.py	14697	1798b88d2383083e 86ca555d82030335 b23ecfc5b32476b5 86a94f0502376be8
web/app.js	39976	87f01233c1a3df78 c5eb2b86c2ddbb64 9cfb08e88031c1ad c133ba83fd850bc0
web/index.html	1870	61c04819e3a3f99c 0b04a64673da53c4 012b753ffa6b03e6 9afc2c60e65ca373
web/styles.css	7013	26e527e38fc8b756 c458ed7ab9889dbf 0539ef29b89da080 7354ed6c50f97c55

Ключевые фрагменты исходного текста

Фрагменты ниже не заменяют manifest/checksum-файлы; они помогают идентифицировать архитектуру, контракты телеметрии и собственные компоненты VAST.

README.md:1-36

```

0001: # VAST Video Analytics Benchmark
0002:
0003: This project scaffolds the experimental study described in exp.txt for the target platform:
0004: - GPU: NVIDIA RTX 3060
0005: - CPU: Intel Core i7-14700K
0006: - RAM: 22 GB

```

```

0007: - SLO deadline: 3 seconds end-to-end latency
0008:
0009: The default `benchmark` mode is strict: publishable runs require native
0010: per-frame telemetry schema v2. Runtime-derived synthetic rows are available
0011: only in explicit `smoke` mode and are excluded from scientific reports.
0012:
0013: It includes:
0014: - Scenario and protocol configuration
0015: - Automated experiment runner
0016: - CPU/GPU metric collection at 1-second frequency
0017: - Repetition handling and structured outputs
0018: - Summary analysis and comparison plots
0019: - Target setup scripts for Linux and Windows/WSL2
0020:
0021: ## Project layout
0022:
0023: - `configs/experiments.yaml`: Hardware target, protocol, structured scenarios, and system command templates
0024: - `configs/datasets.yaml`: Public benchmark and synthetic smoke dataset manifests
0025: - `configs/hosts.yaml`: SSH host inventory used by distributed scenarios
0026: - `scripts/check_system.py`: Prints detected hardware
0027: - `scripts/collect_metrics.py`: CPU/GPU sampler (CSV)
0028: - `scripts/run_experiments.py`: Main matrix execution tool
0029: - `scripts/distributed_executor.py`: SSH/rsync/scp executor for multi-host scenarios
0030: - `scripts/benchmark_adapters.py`: Strict benchmark adapter support matrix and fail-fast contract checks
0031: - `scripts/check_dataset.py`: Dataset checksum validator
0032: - `scripts/train_ql_heft.py`: Seeded offline QL-HEFT policy trainer
0033: - `docs/NATIVE_ADAPTERS.md`: Required native probe and distributed RTP contract
0034: - `scripts/analyze_results.py`: Aggregation and plotting
0035: - `scripts/setup_target.sh`: Linux full-stack bootstrap script
0036: - `scripts/setup_target_windows.ps1`: Windows bootstrap + WSL2 preparation

```

configs/experiments.yaml:1-38

```

0001: schema_version: 2
0002: project:
0003:   name: video-pipeline-scheduler-experiments
0004:   description: Reproducible checkpoint benchmark suite for real KPP AVI video analytics.
0005: benchmark:
0006:   dataset_manifest: configs/datasets.yaml
0007:   default_dataset:
0008:     smoke: smoke_testsrc
0009:     benchmark: kpp_real_h264
0010:   telemetry_schema_version: 2
0011:   publishable_telemetry_sources:
0012:   - native
0013:   scheduler_policies:
0014:   - cpu_only
0015:   - gpu_only
0016:   - static_hybrid
0017:   - heft
0018:   - deadline_aware_heft
0019:   - queue_aware_edf
0020:   - adaptive_weights
0021:   default_seed: 20260323
0022:   active_scenarios:
0023:   - checkpoint_independent_processes_baseline
0024:   - checkpoint_video_dag_shared
0025:   smoke_scenarios:
0026:   - checkpoint_video_dag_shared
0027:   report_scenarios:
0028:   - checkpoint_independent_processes_baseline
0029:   - checkpoint_video_dag_shared
0030:   deadline_ms:
0031:   - 16.7
0032:   - 33.3
0033:   - 50
0034:   - 100
0035:   - 500
0036:   smoke_deadline_ms:
0037:   - 100
0038:   report_deadline_ms:

```

scripts/benchmark_contract.py:14-47

```
0014: import yaml
0015:
0016:
0017: TELEMETRY_SCHEMA_VERSION = 2
0018: FRAME_COLUMNS = [
0019:     "schema_version",
0020:     "run_id",
0021:     "trace_id",
0022:     "stream_id",
0023:     "frame_id",
0024:     "ingress_timestamp_ms",
0025:     "egress_timestamp_ms",
0026:     "e2e_latency_ms",
0027:     "objects",
0028:     "detector",
0029:     "backend",
0030:     "telemetry_source",
0031: ]
0032: FRAME_EVENT_COLUMNS = [
0033:     "schema_version",
0034:     "run_id",
0035:     "trace_id",
0036:     "stream_id",
0037:     "frame_id",
0038:     "stage",
0039:     "role",
0040:     "host",
0041:     "resource",
0042:     "queue_enter_timestamp_ms",
0043:     "stage_start_timestamp_ms",
0044:     "stage_end_timestamp_ms",
0045:     "queue_depth",
0046:     "estimated_cost_ms",
0047:     "policy_action",
```

scripts/run_experiments.py:1123-1156

```
1123:     )
1124:     return expanded
1125:
1126:
1127: def main() -> None:
1128:     parser = argparse.ArgumentParser(description="Run experiment matrix and capture metrics")
1129:     parser.add_argument("--config", default="configs/experiments.yaml")
1130:     parser.add_argument("--systems", nargs="*", default=["all"])
1131:     parser.add_argument("--scenarios", nargs="*", default=["all"])
1132:     parser.add_argument("--repeats", type=int, default=-1)
1133:     parser.add_argument("--measurement", type=int, default=-1, help="Override measurement seconds")
1134:     parser.add_argument("--warmup", type=int, default=-1, help="Override warmup seconds")
1135:     parser.add_argument("--output-root", default="runs")
1136:     parser.add_argument("--hosts-config", type=Path, default=Path("configs/hosts.yaml"))
1137:     parser.add_argument("--mode", choices=["smoke", "benchmark"], default="benchmark")
1138:     parser.add_argument("--dataset", default="")
1139:     parser.add_argument("--datasets", nargs="*", default=None)
1140:     parser.add_argument("--policy", default="static_hybrid")
1141:     parser.add_argument("--policies", nargs="*", default=None)
1142:     parser.add_argument("--deadline-modes", nargs="*", default=["all"])
1143:     parser.add_argument(
1144:         "--run-kind",
1145:         choices=["auto", "local", "heterogeneous", "single-server-distributed", "distributed"],
1146:         default="auto",
1147:     )
1148:     parser.add_argument("--local-only", action="store_true", help="Deprecated alias for --run-kind local")
1149:     parser.add_argument("--single-server-host", default="127.0.0.1")
1150:     parser.add_argument("--single-server-port", type=int, default=22)
1151:     parser.add_argument("--single-server-user", default="")
1152:     parser.add_argument("--seed", type=int, default=None, help="Base seed shared across systems for a scenario/repeat")
1153:     parser.add_argument("--dry-run-plan", action="store_true")
1154:     parser.add_argument("--continue-on-error", action="store_true")
1155:     parser.add_argument(
1156:         "--resume-run-root",
```

scripts/vast_gui.py:1053-1080

```
1053:
1054:
1055: def make_handler(app: VastGuiApp) -> type[BaseHTTPRequestHandler]:
1056:     class VastRequestHandler(BaseHTTPRequestHandler):
1057:         server_version = "VASTGUI/1.0"
1058:
1059:         def log_message(self, fmt: str, *args: Any) -> None:
1060:             sys.stderr.write("[vast-gui] " + fmt % args + "\n")
1061:
1062:         def read_json(self) -> dict[str, Any]:
1063:             length = int(self.headers.get("Content-Length", "0") or "0")
1064:             if length <= 0:
1065:                 return {}
1066:             raw = self.rfile.read(length).decode("utf-8")
1067:             try:
1068:                 payload = json.loads(raw)
1069:             except json.JSONDecodeError as exc:
1070:                 raise GuiError(f"invalid JSON: {exc}") from exc
1071:             if not isinstance(payload, dict):
1072:                 raise GuiError("request body must be a JSON object")
1073:             return payload
1074:
1075:         def send_json(self, status: int, payload: dict[str, Any]) -> None:
1076:             json_response(self, status, payload)
1077:
1078:         def handle_error(self, exc: Exception) -> None:
1079:             if isinstance(exc, GuiError):
1080:                 self.send_json(exc.status, {"ok": False, "error": str(exc)})
```

deploy/gstreamer_adaptivescheduler/gstadaptivescheduler.c:291-304

```
0291: static gboolean plugin_init(GstPlugin* plugin) {
0292:     return gst_element_register(plugin, "adaptivescheduler", GST_RANK_NONE, GST_TYPE_ADAPTIVE_SCHEDULER);
0293: }
0294:
0295: GST_PLUGIN_DEFINE(
0296:     GST_VERSION_MAJOR,
0297:     GST_VERSION_MINOR,
0298:     adaptivescheduler,
0299:     "VAST adaptive scheduler custom GStreamer element",
0300:     plugin_init,
0301:     "1.0.0",
0302:     "LGPL",
0303:     "VAST",
0304:     "https://example.invalid/vast")
```

deploy/native_gst_probe/vast_native_gst_probe.cpp:451-484

```
0451:     auto handle_buffer = [&](GstBuffer* buffer) {
0452:         if (buffer == nullptr) {
0453:             return;
0454:         }
0455:         GstRTPBuffer read_rtp = GST_RTP_BUFFER_INIT;
0456:         if (!gst_rtp_buffer_map(buffer, GST_MAP_READ, &read_rtp)) {
0457:             return;
0458:         }
0459:         const bool marker = gst_rtp_buffer_get_marker(&read_rtp);
0460:         gst_rtp_buffer_unmap(&read_rtp);
0461:
0462:         Trace trace;
0463:         bool completed_frame = false;
0464:         {
0465:             std::lock_guard<std::mutex> lock(self->mutex_);
0466:             StreamState& state = self->states_[static_cast<std::size_t>(ctx->stream_id)];
0467:             if (!state.has_output_trace) {
0468:                 state.current_output_trace.stream_id = static_cast<std::uint8_t>(ctx->stream_id);
```

```
0469:         state.current_output_trace.frame_id = state.edge_frame_id++;
0470:         state.current_output_trace.ingress_ms = now_ms();
0471:         state.has_output_trace = true;
0472:     }
0473:     trace = state.current_output_trace;
0474:     if (marker) {
0475:         state.has_output_trace = false;
0476:         completed_frame = true;
0477:     }
0478: }
0479:
0480: GstRTPBuffer write_rtp = GST_RTP_BUFFER_INIT;
0481: if (!gst_rtp_buffer_map(buffer, GST_MAP_READWRITE, &write_rtp)) {
0482:     return;
0483: }
0484: const auto payload = pack_trace(trace);
```